

Introduction to Regular Expressions

Pattern Matching and Text Processing

What is Regular Expression?

- A regular expression (regex) is a sequence of characters that defines a search pattern.
- Used for string searching and manipulation.
- Helps in validation, parsing, and text processing.
- Python provides 're' module to work with regex.

Why use Regular Expressions?

- Validate input formats (emails, phone numbers).
- Extract specific data from large text.
- Replace or split text efficiently.
- Useful in data cleaning and web scraping.

Special Symbols (Metacharacters)

- `.` → Matches any character except newline.
- `^` → Matches beginning of a string.
- `$` → Matches end of a string.
- `*` → 0 or more repetitions.
- `+` → 1 or more repetitions.
- `?` → 0 or 1 occurrence.
- `{n}` → Exactly n times.
- `{n,}` → At least n times.
- `{n,m}` → Between n and m times.

1. . → Matches any character except newline

- Matches any single character between a and c. (a\nc not matched because . does not match newline).

```
1  import re
2  print(re.findall(r"a.c", "abc a1c a-c acc axc a\nc"))
3  
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Temp\te
['abc', 'a1c', 'a-c', 'acc', 'axc']
PS C:\Users\Acer>
```

2. ^ → Matches beginning of a string

- Only matches when "Hello" is at the **start** of string.

```
1  import re
2  print(re.findall(r"^Hello", "Hello World"))
3  print(re.findall(r"^Hello", "Hi Hello"))
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Lo
['Hello']
[]
PS C:\Users\Acer>
```

3. \$ → Matches end of a string

- Matches "World" only if it is at the **end**.

```
1  import re
2  print(re.findall(r"World$", "Hello World"))
3  print(re.findall(r"World$", "World is big"))
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\T
['World']
[]
PS C:\Users\Acer>
```

4. * $\rightarrow$ 0 or more repetitions

- "b" is repeated 0, 1, 2, 3... times after "a".

```
1  import re
2  print(re.findall(r"ab*", "a ab abb abbb abc"))
3  |
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Tei
['a', 'ab', 'abb', 'abbb', 'ab']
PS C:\Users\Acer>
```


5. + $\rightarrow$ 1 or more repetitions

- Must have at least **one** "b".

```
1  import re
2  print(re.findall(r"ab+", "a ab abb abbb abc"))
3  
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Temp\t
['ab', 'abb', 'abbb', 'ab']
PS C:\Users\Acer>
```

6. ? → 0 or 1 occurrence

- "b" is **optional** (0 or 1 time).

```
1  import re
2  print(re.findall(r"ab?", "a ab abb abbb abc"))
3
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Ter
['a', 'ab', 'ab', 'ab', 'ab']
PS C:\Users\Acer>
```

7. {n} → Exactly n times

- Matches "a" followed by exactly **2 b's**.

```
1  import re
2  print(re.findall(r"ab{2}", "ab abb abbb abbbb"))
3  
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Temp\
['abb', 'abb', 'abb']
PS C:\Users\Acer>
```

8. {n,} → At least n times

- Matches "a" followed by **2 or more b's**.

```
1  import re
2  print(re.findall(r"ab{2,}", "ab abb abbb abbbb"))
3
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Temp\t
['abb', 'abbb', 'abbbb']
```

```
PS C:\Users\Acer>
```

9. $\{n,m\} \rightarrow$ Between n and m times

- Matches "a" followed by **1 to 3 b's**.

```
1  import re
2  print(re.findall(r"ab{1,3}", "ab abb abbb abbbb"))
3  
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Temp\tempCc
['ab', 'abb', 'abbb', 'abbb']
PS C:\Users\Acer>
```

Operator	Explanation	Example	Output
.	Matches any character except newline	<code>re.findall(r"a.c", "abc a1c a-c acc axc a\nc")</code>	<code>['abc', 'a1c', 'a-c', 'acc', 'axc']</code>
^	Matches beginning of a string	<code>re.findall(r"^ab", "abc ab abcd")</code>	<code>['ab']</code>
\$	Matches end of a string	<code>re.findall(r"ab\$", "cab dab ab")</code>	<code>['ab']</code>
*	Matches 0 or more repetitions	<code>re.findall(r"ab*", "a ab abb abbb abc")</code>	<code>['a', 'ab', 'abb', 'abbb', 'ab']</code>
+	Matches 1 or more repetitions	<code>re.findall(r"ab+", "a ab abb abbb abc")</code>	<code>['ab', 'abb', 'abbb', 'ab']</code>
?	Matches 0 or 1 occurrence	<code>re.findall(r"ab?", "a ab abb abbb abc")</code>	<code>['a', 'ab', 'ab', 'ab', 'ab']</code>
{n}	Matches exactly n times	<code>re.findall(r"a{2}", "a aa aaa aaaa")</code>	<code>['aa', 'aa']</code>
{n,}	Matches at least n times	<code>re.findall(r"a{2,}", "a aa aaa aaaa")</code>	<code>['aa', 'aaa', 'aaaa']</code>
{n,m}	Matches between n and m times	<code>re.findall(r"a{2,3}", "a aa aaa aaaa")</code>	<code>['aa', 'aaa', 'aaa']</code>

Character Classes

- `[]` → Match any one character inside brackets.
- `[^ ]` → Match any character *not* inside brackets.
- `\d` → Any digit (0–9).
- `\D` → Any non-digit.
- `\w` → Word character (letters, digits, underscore).
- `\W` → Non-word character.
- `\s` → Whitespace.
- `\S` → Non-whitespace.

Regex	Meaning	Real-Life Example	Regex Pattern	Matched Example (Python Output)
[...]	Match any one character inside brackets	Validate vowels in a name	r"[aeiou]"	"Neelkanth" → ['e', 'e', 'a']
[^...]	Match any character not inside brackets	Extract consonants (not vowels)	r"[^aeiou]"	"Neelkanth" → ['N','l','k','n','t','h']
\d	Any digit (0–9)	Find all numbers in an address	r"\d"	"Flat 23, Block 7" → ['2','3','7']
\D	Any non-digit	Extract only text from an address	r"\D+"	"Flat 23" → ['Flat ']
\w	Any word character (A–Z, a–z, 0–9, _)	Validate username (letters, digits, underscore)	r"\w+"	"user_123" → ['user_123']
\W	Any non-word character	Split text by spaces/punctuation	r"\W+"	"Hello, World!" → [, ' , ' !']
\s	Any whitespace (space, tab, newline)	Find spaces between words	r"\s"	"New Delhi" → [' ']
\S	Any non-whitespace	Extract actual words (ignore spaces)	r"\S+"	"New Delhi" → ['New', 'Delhi']

1. [] → Match any one character inside brackets

Matches any vowel (a, e, i, o, u).

```
1  import re
2  print(re.findall(r"[aeiou]", "hello world"))
3
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Temp
['e', 'o', 'o']
PS C:\Users\Acer>
```

2. [^] → Match any character NOT inside brackets

Matches all characters except vowels.

```
2  import re
3  print(re.findall(r"[^aeiou]", "hello world"))
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\`
['h', 'l', 'l', ' ', 'w', 'r', 'l', 'd']
```

```
PS C:\Users\Acer>
```

3. \d → Any digit (0–9)

Extracts digits.

```
1  import re
2  print(re.findall(r"\d", "Room 123, Floor 4"))
3
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Te
['1', '2', '3', '4']
```

```
PS C:\Users\Acer>
```

4. \D → Any non-digit

Extracts everything that is not a digit.

```
1 import re
2 print(re.findall(r"\D", "Room 123, Floor 4"))
3
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Temp\tempCode
['R', 'o', 'o', 'm', ' ', ',', ' ', 'F', 'l', 'o', 'o', 'r', ' ']
```

```
PS C:\Users\Acer>
```

5. \w → Word character (letters, digits, underscore)

Matches letters, digits, and _.

```
1 import re
2 print(re.findall(r"\w", "hello_123 world!"))
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Temp\tempCodeRu
['h', 'e', 'l', 'l', 'o', '_', '1', '2', '3', 'w', 'o', 'r', 'l', 'd']
PS C:\Users\Acer>
```

6. \W → Non-word character

Matches spaces, punctuation, etc.

```
1  import re
2  print(re.findall(r"\W", "hello_123 world!"))
3  
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Te
[' ', '!']
```

```
PS C:\Users\Acer>
```

7. \s → Whitespace

Matches space or tab.

```
1  import re
2  print(re.findall(r"\s", "hello world\tPython"))
3  
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Temp\
[' ', '\t']
PS C:\Users\Acer>
```

8. \S → Non-whitespace

Matches everything except spaces /tabs /newlines.

```
1 import re
2 print(re.findall(r"\S", "hello world\tPython"))
3
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Temp\tempCodeRunnerFile.python"
['h', 'e', 'l', 'l', 'o', 'w', 'o', 'r', 'l', 'd', 'P', 'y', 't', 'h', 'o', 'n']
PS C:\Users\Acer>
```


8. \S → Non-whitespace

Matches everything except spaces /tabs /newlines.

```
1 import re
2 print(re.findall(r"\S", "hello world\tPython"))
3
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Temp\tempCodeRunnerFile.python"
['h', 'e', 'l', 'l', 'o', 'w', 'o', 'r', 'l', 'd', 'P', 'y', 't', 'h', 'o', 'n']
PS C:\Users\Acer>
```

Grouping & Alternation

- $() \rightarrow$ Groups expressions.
- $| \rightarrow$ OR operator.

1. () → Groups expressions

- Used to group parts of the pattern.
- Helps in capturing sub-patterns and applying quantifiers to the group as a whole.

- (ab)+ means "ab" repeated **1 or more times** as a unit.
- Matches: "ab", "abab", "ababab".

```
1  import re
2
3  # Example 1: Grouping
4  text = "ababab ab abc"
5  result = re.findall(r"(ab)+", text)
6  print(result)
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

```
PS C:\Users\Acer> python -u "C:\Users\Acer\Ap
['ab', 'ab', 'ab']
PS C:\Users\Acer>
```

- `(\d{4})` → captures the year
- `(\d{2})` → captures month
- `(\d{2})` → captures day

```
1  import re
2  # Example 2: Capturing groups
3  text = "2025-08-29"
4  result = re.findall(r"(\d{4})-(\d{2})-(\d{2})", text)
5  print(result)
6  
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Temp\te
[('2025', '08', '29')]
PS C:\Users\Acer>
```

2. | → OR operator

Works like **logical OR** between alternatives.

```
1  import re
2  # Example 1: OR operator
3  text = "cat dog rat bat"
4  result = re.findall(r"cat|dog", text)
5  print(result)
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

Py

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Microsoft\Windows\Apps\python.exe"
['cat', 'dog']
PS C:\Users\Acer>
```

Matches either "cat" OR "dog".

2. | → OR operator

Works like **logical OR** between alternatives.

```
1  import re
2  # Example 2: OR with groups
3  text = "color colour colr"
4  result = re.findall(r"colou?r", text)
5  print(result)
6
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\
['color', 'colour']
PS C:\Users\Acer>
```

- "u?" means u is optional
- So it matches "color" OR "colour".

Python re Module

Common functions:

- `re.match(pattern, string)` → Match only at beginning.
- `re.search(pattern, string)` → Search anywhere in string.
- `re.findall(pattern, string)` → Find all matches.
- `re.sub(pattern, repl, string)` → Replace matches.
- `re.split(pattern, string)` → Split string.

1. re.match(pattern, string)

Matches only **at the beginning** of the string.

- **Real-life use case:** Validate that a filename starts with "IMG" before processing images.

```
1  import re
2  # Example: Checking if a filename starts with "IMG"
3  text = "IMG_2025_photo.jpg"
4  result = re.match(r"IMG", text)
5  print(result)
6
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Temp'
<re.Match object; span=(0, 3), match='IMG'>
```

```
PS C:\Users\Acer>
```

2. re.search(pattern, string)

Searches anywhere in the string.

Real-life use case: Searching logs for error keywords.

```
1 import re
2
3 # Example: Searching if the string contains "error"
4 log = "2025-08-30 10:00:00 - Critical error occurred in system"
5
6 result = re.search(r"error", log)
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Temp\tempCodeRunne
error
```

```
PS C:\Users\Acer>
```

3. re.findall(pattern, string)

Finds **all occurrences** of the pattern.

Real-life use case: Extracting order IDs, phone numbers, or account numbers from text.

```
1  import re
2
3  # Example: Extracting all numbers from text
4  text = "Order IDs: 101, 205, 333, 450"
5
6  result = re.findall(r"\d+", text)
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Lo
['101', '205', '333', '450']
```

```
PS C:\Users\Acer>
```

4. re.sub(pattern, repl, string)

Replaces matches with something else.

Real-life use case: Cleaning messy text (extra spaces, unwanted words).

```
1  import re
2
3  # Example: Replacing multiple spaces with a single space
4  text = "Hello      World    from   Python"
5
6  result = re.sub(r"\s+", " ", text)
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Temp\tempCc
Hello World from Python
PS C:\Users\Acer>
```

5. re.split(pattern, string)

Splits string by regex pattern.

Real-life use case: Splitting CSV or mixed delimiter files.

```
1  import re
2
3  # Example: Splitting by commas or semicolons
4  text = "apple,banana;grape;orange,melon"
5
6  result = re.split(r"[,;]", text)
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local
['apple', 'banana', 'grape', 'orange', 'melon']
PS C:\Users\Acer>
```

Function	Explanation	Example Code	Output
re.match(pattern, string)	Checks match only at the beginning of the string.	python import re print(re.match(r"hello", "hello world"))	<re.Match object; span=(0, 5), match='hello'>
re.search(pattern, string)	Searches anywhere in the string; returns first match.	python import re print(re.search(r"world", "hello world"))	<re.Match object; span=(6, 11), match='world'>
re.findall(pattern, string)	Returns all non-overlapping matches as a list.	python import re print(re.findall(r"\d+", "Order 23, Bill 456, Code 89"))	['23', '456', '89']
re.sub(pattern, repl, string)	Replaces matches with a replacement string.	python import re print(re.sub(r"\d+", "#", "Room 23, Floor 5"))	"Room #, Floor #"
re.split(pattern, string)	Splits the string by the matched pattern.	python import re print(re.split(r"\s+", "Python Regex Module"))	['Python', 'Regex', 'Module']